IUT Robert Schuman
Institut universitaire de technologie
Université de Strasbourg

X62 - Optimisation pour l'aide à la décision
TP n° 4
Réseaux de neurones

1 Données MNIST

L'ensemble de données MNIST (Modified National Institute of Standards and Technology) est l'un des ensembles de données les plus emblématiques dans le domaine de l'apprentissage automatique. Il s'agit d'une collection de chiffres manuscrits utilisée pour entraîner divers systèmes de reconnaissance d'images. Dans ce TP, nous allons utiliser ces données afin d'essayer de résoudre un problème classique de classification.

L'ensemble de données MNIST contient 42000 images. Chacune est un carré de 28×28 pixels, en niveaux de gris, représentant un chiffre manuscrit entre 0 et 9. Les images sont stockées sous forme de matrices 28×28, où chaque élément de la matrice correspond à l'intensité d'un pixel (entre 0 et 255).

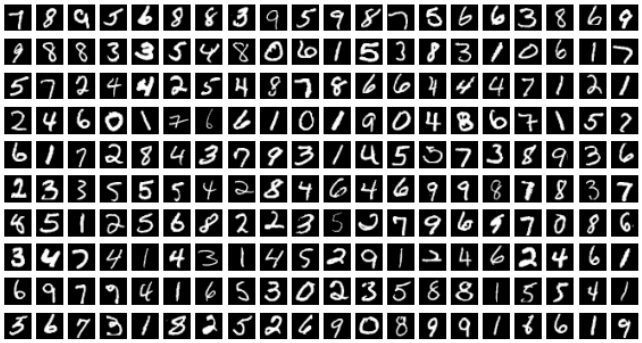


FIGURE 1 – Exemples d'images de l'ensemble de données MNIST

Les données sont disponibles dans le fichier `data_mnist.zip` (sur Moodle). Pour en extraire les données, il est possible d'utiliser la fonction `read_csv` de la librairie `pandas`. La fonction renvoie un objet de type `Dataframe`.

```
    data = pd.read_csv('data_mnist.zip')
```

Chaque image est représentée sur une ligne, avec tout d'abord son 'label' (sa classe), puis les pixels la composant. En effet, elles ont été transformées en vecteur de 784 valeurs de pixels par une opération 'flatten'. Les valeurs des pixels sont comprises entre 0 et 255.

```
   label  pixel0  pixel1  pixel2  ...  pixel780  pixel781  pixel782  pixel783
0      1       0       0       0  ...         0         0         0         0
1      0       0       0       0  ...         0         0         0         0
2      1       0       0       0  ...         0         0         0         0
3      4       0       0       0  ...         0         0         0         0
4      0       0       0       0  ...         0         0         0         0

[5 rows x 785 columns]
```

Pour faciliter la manipulation des données lors de l'apprentissage, il est préférable d'utiliser des matrices `numpy` :

- X, contenant uniquement les données (valeurs des pixels pour chaque image du jeu de données) ;
- y, contenant les labels (classes).

Nous nous intéressons pour le moment à un problème de classification binaire, donc il n'est pas utile de charger l'ensemble des images du fichier, mais uniquement celles de deux classes (les zéros et les uns par exemple). Commencez par écrire une fonction :

```
def load_mnist(a, b, nnums=-1):
```

qui récupère `nnums` images dans le fichier `data_mnist.zip` des classes `a` et `b`. Si `nnums` est égal à `-1`, toutes les images des deux classes sont chargées.

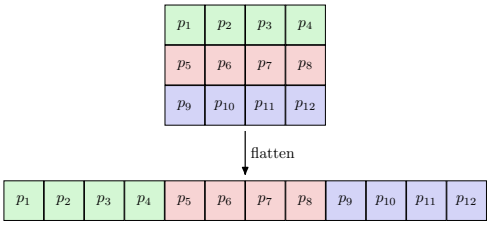


FIGURE 2 – Exemple d'image 4×3 applatie en vecteur de longueur 12

On peut visualiser un exemple de manière simple, en redonnant à un vecteur (représentant une image) sa taille d'origine 28×28 puis en utilisant `imshow` de la librairie `matplotlib` :

```
import matplotlib.pyplot as plt
# on prend un élément de X
example = X[3]
# on le transforme en vecteur
example = example.flatten()
# on le redimensionne en 28x28
example.resize((28, 28))
# on affiche ses valeurs sous forme d'une image en niveaux de gris
plt.imshow(example, cmap='gray')
plt.axis('off')
plt.show()
```

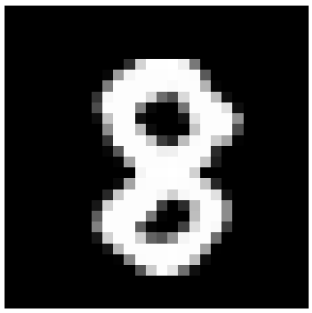


FIGURE 3 – Visualisation d'un chiffre de la classe '8' de l'ensemble de données MNIST

2 Classification binaire pour distinguer deux chiffres

Pour commencer, l'idée est de tester le perceptron développé précédemment pour effectuer une classification binaire. L'objectif est d'essayer de distinguer les chiffres 1 des 0 par exemple.

Préparation des données Charger les données, ne conserver que les éléments des classes 0 et 1 puis les transformer en matrices `numpy` (cf. section précédente). Enfin, normaliser les valeurs des pixels, afin qu'elles soient entre 0 et 1.

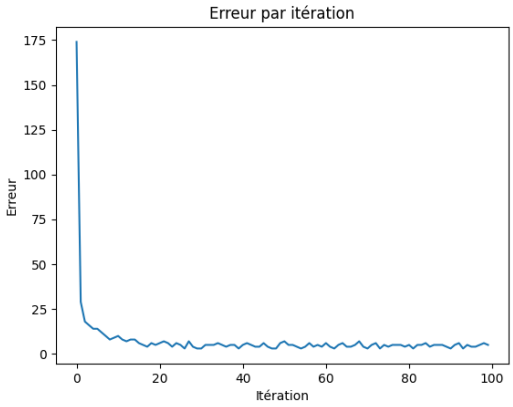
Jeux d'entraînement et de test Afin de pouvoir évaluer la qualité de classification du perceptron entraîné sur cette tâche, le jeu de données peut être séparé en deux : un jeu pour l'entraînement et un autre conservé uniquement pour l'évaluation une fois le perceptron entraîné. Cela signifie que la performance de classification du perceptron va être évaluée sur des données n'ayant jamais été utilisées durant l'apprentissage. Cette séparation garantie que le modèle représenté par le perceptron est capable de généraliser à des données inconnues et n'a pas *appris par cœur* les données qu'on lui a présentées durant l'apprentissage.

Pour séparer aléatoirement le jeu de données en jeu d'entraînement et jeu de test, il est possible d'utiliser la fonction `train_test_split` de la librairie `scikit-learn` :

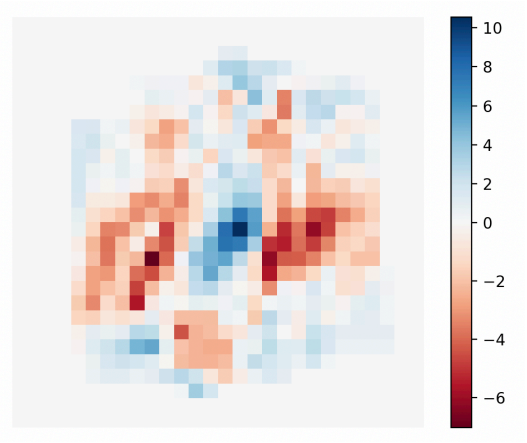
```
from sklearn.model_selection import train_test_split

# séparation des données 75% training et 25% test avec ordonnancement aléatoire
train_data, test_data = train_test_split(data, test_size=0.25,
                                         random_state=1, shuffle=True)
```

Entraînement du perceptron Utiliser le perceptron défini précédemment et l'entraîner sur les données d'entraînement. Les hyperparamètres sont à ajuster en fonction des données. Vérifier la convergence sur la courbe d'erreur.



Visualisation des poids du perceptron Il est possible de visualiser les valeurs des poids (sous forme d'image) afin de comprendre (un peu) quels pixels utilise principalement le perceptron pour donner sa réponse :



Évaluation du modèle Le modèle entraîné peut maintenant être évalué en fonction des réponses qu'il va donner sur les données conservées dans le jeu de test. La précision du modèle se calcule en faisant le rapport entre le nombre de fois où le perceptron donne la bonne réponse sur le nombre total d'éléments dans le jeu de données. Une méthode

```
    def calculate_accuracy(self, x_test, y_test):
```

peut être ajoutée à la classe `Perceptron` pour effectuer ce calcul.

3 Influence de la fonction de coût : quadratique vs entropie croisée

Avant de généraliser le modèle au cas multiclasse, il peut être intéressant d'étudier l'impact du choix de la fonction de coût (ou *loss function*) sur la qualité et la vitesse de l'apprentissage.

Pour cela, il faut modifier le perceptron binaire (distinguant par exemple les 0 des 1) en remplaçant la fonction d'activation à seuil (Heaviside) par une fonction sigmoïde (voir cours M61).

Question 1 - Modifier le code du perception pour utiliser une activation sigmoïde et l'erreur quadratique moyenne (*mean square error*) comme fonction de coût. Entraîner ce modèle pour qu'il reconnaisse les 0 du reste des chiffres.

Question 2 - Faire de même avec la fonction de coût d'entropie croisée (*cross-entropy*).

Question 3 - Comparer les courbes d'erreur en fonction des époques pour les deux modèles, initialisés avec des poids identiques et un learning rate identique. Peut-on observer une différence significative dans la vitesse de convergence, notamment lors des premières itérations ? Le résultat théorique est-il confirmé par la pratique ?

4 Passage au multiclasse

Afin de passer à la classification en 10 classes des images du jeu de données MNIST, il est possible de créer un réseau de neurones pour obtenir un modèle avec une expressivité suffisante pour représenter ce problème plus complexe que la classification binaire.

Commencer par créer un réseau simple avec 10 neurones calculant une combinaison linéaire des entrées ($z = Wx + b$) suivis de la fonction *softmax* pour obtenir une probabilité d'appartenance à chacune des 10 classes

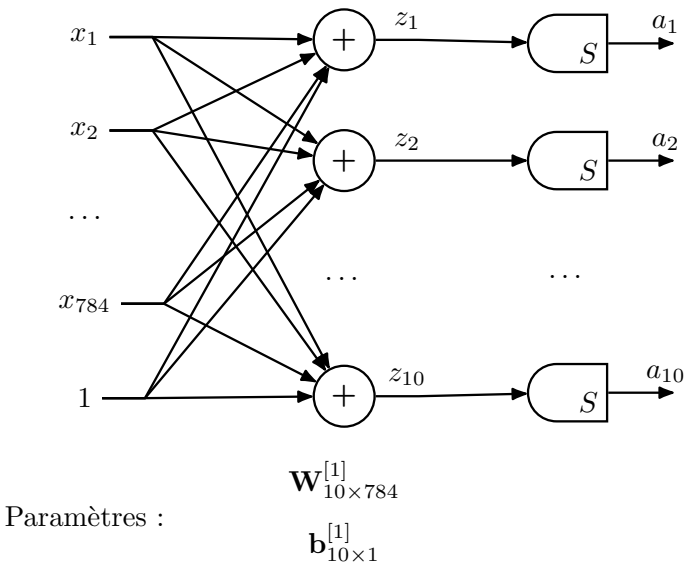


FIGURE 4 – Architecture du réseau de neurones avec S la fonction *softmax*

Question 4 - Écrire une classe avec un constructeur et une méthode d'initialisation des poids du réseau en fonction de `d` et `n` représentant respectivement la taille de l'entrée (ici 784 sur les données MNIST) et la taille de la couche de neurones (ici 10, le nombre de classes). Les poids des neurones et le biais sont initialisés à 0.

```
class NeuralNet:
    def __init__(self):
        ''' constructeur de la classe '''

    def initialize_parameters(self, d, n):
        ''' initialisation des poids du réseau '''
```

Question 5 - Ajouter une méthode de propagation, qui calcule la sortie du réseau pour une observation en entrée `x`.

```
    def forward_propagation(self, x):
        ''' calcul de la sortie pour une observation x '''
```

One-hot encoding Le *one-hot encoding* transforme une variable catégorielle ayant n classes possibles en un vecteur de taille n où un seul élément est "chaud" (c'est-à-dire, mis à 1) et tous les autres éléments sont "froids" (c'est-à-dire, mis à 0). Chaque position dans le vecteur représente une classe distincte.
Exemple : sur les données MNIST, un chiffre de la classe '4' aura un vecteur de sortie attendu de la forme :

$$(0,0,0,0,1,0,0,0,0,0,0)$$

Les sorties attendues pour chaque image sont transformées en *one-hot* pour faciliter la comparaison avec la sortie du réseau (un vecteur de probabilités d'appartenance aux classes).

Fonction de coût Pour la classification multiclasse, où les étiquettes sont codées en *one-hot* et les prédictions sont un vecteur de probabilités pour chaque classe, le coût de log-vraisemblance est adapté (voir le support de M61 à ce sujet).

Question 6 - Coder cette fonction afin de pouvoir afficher l'évolution de la fonction de coût lors de l'apprentissage :

```
def cost_function(self, y, y_pred):
    ''' calcul de la fonction de coût pour un résultat y_pred du réseau '''
```

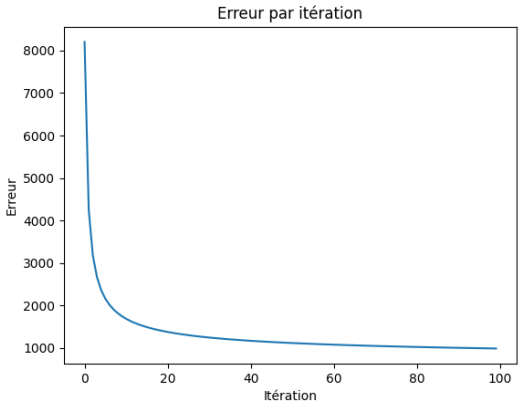


FIGURE 5 – Évolution de la fonction de coût au cours de l'apprentissage

Rétropropagation et mise à jour des poids Une fois la sortie calculée ainsi que la fonction de coût, il est possible de calculer les gradients pour l'adaptation des poids de la couche de neurones ainsi que le biais.

Question 7 - Écrire la fonction de rétropropagation du gradient :

```
def backward_prop(self, x, y):
    ''' calcul des gradients et mise à jour des poids '''
```

Question 8 - Écrire la méthode réalisant l'apprentissage pour un jeu de données (x, y) :

```
def train(self, x, y):
    ''' algorithme d'entraînement '''
```

4.1 Évaluation de la précision du modèle

On peut maintenant s'intéresser aux performances de généralisation du modèle, en le testant avec les données du jeu de test.

Question 9 - Écrire une méthode calculant la précision moyenne du réseau sur un jeu de données passé en paramètre. Rappel : il suffit de calculer le pourcentage de bonnes réponses sur les données de test.

```
def accuracy(self, x_test, y_test):
    ''' calcul de la précision pour une observation input en fonction de son label '''
```

4.2 Améliorations possibles

Question 10 - Modifier les hyperparamètres (learning rate, nombre d'itérations) de l'apprentissage peut permettre d'améliorer grandement les performances du modèle.

Pour accélérer la convergence, il est aussi possible de procéder par mini-batches comme pour la régression vue au TP2.

Question 11 - Ajouter une couche cachée peut permettre d'avoir L'idée est de coder une classe pour représenter un réseau de ce type (une seule couche cachée), on stockant les poids et biais de la couche cachée et de la couche de sortie dans des matrices (`W1`, `b1`, `W2`, `b2`). La taille de la couche cachée sera un paramètre de la classe afin de pouvoir être facilement adaptée. D'autres paramètres sont à configurer comme le *learning rate* ou le nombre maximal d'époques d'entraînement.

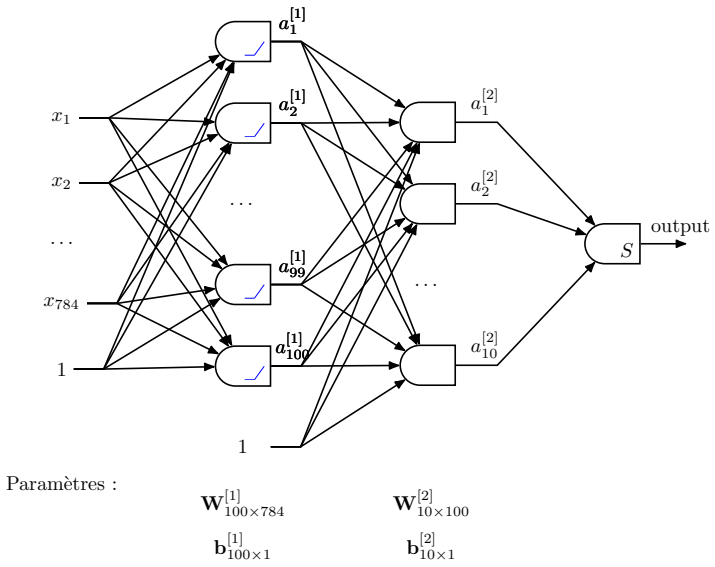


FIGURE 6 – Architecture du réseau de neurones avec une couche cachée de 100 neurones avec une fonction d'activation $ReLu$